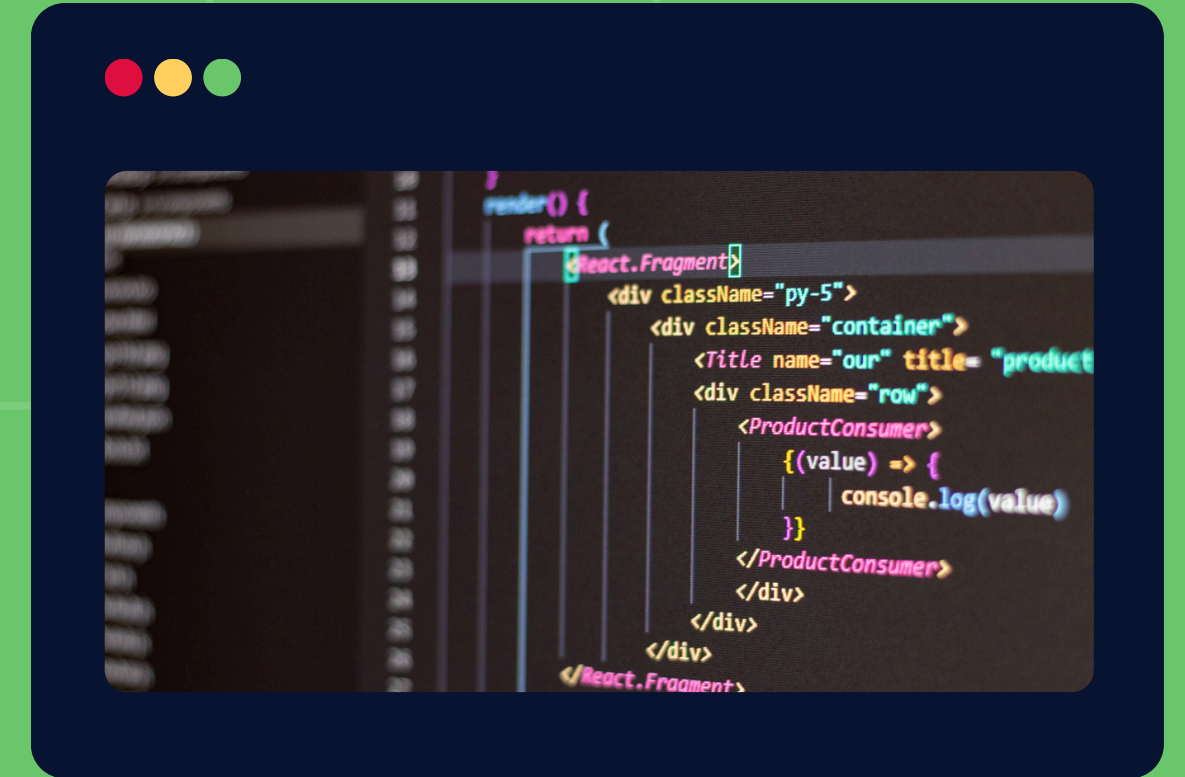


SC2002 Group Assignment

Internship Management System



Done by SCEC Group 4:
Yasu, Advait, Sahana, Jolie

Introduction



Goal

An application that allows students, company representatives, and career center staff to manage internship postings, applications, and approvals efficiently



Our Approach Taken

- **Application Type:** Java Command Line Interface (CLI) system with a clear separation of user roles
- Applied object-oriented design principles for structure and clarity
- Used CSV storage for simple and portable data persistence
- Design focuses on simplicity, modularity and future extensibility



Assumptions made

- **Single-user system**, interaction is done via console menus
- Limited number of applications/internships based on **predefined constants**
- System prioritizes **clarity and maintainability** over performance.
- User accounts for Students and Career Center Staff are not created dynamically (managed via CSV edits)
- Intended for small to medium scale usage

Design Considerations



Low Coupling

- Modules interact via clear, minimal interfaces, reducing interdependence.
- Changes in one component have minimal impact on others, improving maintainability.
- Achieved by separating concerns into Entity, Controller, and Boundary/UI classes.



High Cohesion

- Each class has a focused, single responsibility for clarity and reusability.
- Enhances readability, reduces bugs, and simplifies maintenance.

This achieves...

- **Extensibility**
- **Maintainability**

Our Design Patterns



Entity-Control-Boundary (ECB) Architectural Pattern

- **Entity classes : Core data models**
 - Student
 - Internship
- **Control classes: Manage workflow**
 - CompanyRepController
 - InternshipController
 - CareerCenterStaffController
- **Boundary classes : System interface**
 - CompanyRepMenuDisplay
 - CareerStaffMenuDisplay



Component Based Menus

- Menus encapsulated in separate classes (e.g. CompanyRepMenuDisplay) to **modularize UI**
- User classes (e.g. CompanyRepresentative) delegate menu rendering and flow to menu components instead of handling UI directly
- Ensures **clear separation of concerns**
- Menus are **reusable and extensible**

SOLID Principles Used

●●● Single Responsibility Principle (SRP)

- Student: manages student-specific data and behaviors
- Internship: responsible for internship attributes
- CompanyRepresentative & CareerCenterStaff: manage their own user roles and related functions
- InternshipManagementApp: central coordinator/controller

●●● Liskov Substitution Principle (LSP)

- User abstract class → subclasses: Student, CompanyRepresentative, CareerCenterStaff
- Controller treats users uniformly as User objects → invokes displayMenu()
- All subclasses adhere to expected behavior defined by User

●●● Dependency Inversion Principle (DIP)

- High-level logic depends on the User abstraction, not concrete classes.
- New user types can be added without modifying existing code.
- Reduces dependency on concrete details, enhancing maintainability.

●●● Open/Closed Principle (OCP)

- User class → basic attributes (e.g., userID)
- CompanyRepresentative → internships created by them
- Student → appliedInternships storing registered internships
- New user classes can inherit User without changing existing classes

●●● Interface Segregation Principle (ISP)

- User class has only essential methods, keeping roles focused.
- Subclasses override relevant methods, avoiding unnecessary functionality.
- New user types can be added without changing existing classes.

OOP Design Principles



Abstraction

- The process of simplifying complex ideas by focusing on essential features
- An abstract class User encapsulates shared properties and behaviour for all user types
- Abstract method enforce user specific implementations



Encapsulation / Information Hiding

- Encapsulation : keep important fields private or protected
 - access these using get methods
 - prevents other classes for modifying sensitive data directly
- Information Hiding :
- keeps system secure , organized and easy to maintain



Inheritance

- New classes inherit attributes and methods, reducing code duplication.
- Child classes, like Student and Staff, inherit from the User class, reusing common methods (e.g., getID()).
- Simplifies development by reusing code, cutting down on effort and length.



Polymorphism

- allows different objects to be treated as objects of a common type
- subclasses can override abstract class methods to show its own role-specific methods and customise the behaviours
- makes the system flexible

Additional Features Added



Notifications

Get notified when
application gets approved
No need of manual
checking.



Bookmark

Save and Revisit
internships,
No need of using a
notebook to note them





Live Demo



Thank You!